PaCR: An Agentic Preference-aligned TAP Conflict Handling

Ronghua Li, Zi Liang, Yaxin Xiao, Qingqing Ye, *Member, IEEE*, Haibo Hu, *Senior Member, IEEE*

***Abstract*—With the increasing complexity of smart home environments, Trigger-Action-Programming (TAP) conflicts have emerged as critical issues, occurring when multiple automation apps compete for control over shared devices or interact in unintended ways. While existing approaches detect and resolve such conflicts using pre-defined rules or expert knowledge, they fail to account for the significant variability in individual preferences for conflict resolution and significantly degrade automation system utility. To address this gap, we propose PaCR (Preference-aligned Conflict Resolver), a novel agentic framework that resolves conflicts while aligning with users' personalized preferences for automation behavior. PaCR employs a two-step conflict resolution process: first, it interactively gathers contextual information from smart home configurations to generate conflict resolution agents with user profiles that accurately depict user preferences and characteristics; then, the agents acting as users utilize semantic reasoning capabilities of LLMs to assist in generating personalized conflict detection and resolution strategies. Due to the agentic confict resolving feature, we evaluate the effectiveness of PaCR with existing datasets at scale. PaCR demonstrates strong capabilities in addressing conflicts based on users' need and significantly improves conflict resolution efficacy and interpretability, offering a user-centric approach to managing smart home automation conflicts.**

***Index Terms*—Smart Home automation, automation conflict, rule modification, personal variance alignment, end-user interaction**

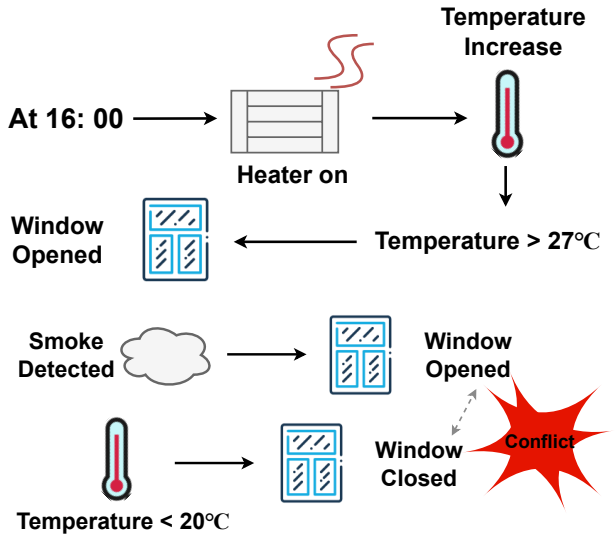

Fig. 1: Examples of interaction threats and conflicts. **Interaction:** A time-triggered app turns on the heater, which increases indoor temperature and interacts with another app that opens the window [5]; **Conflict:** An app opens the window for smoke evacuation, but another app closes the window due to a temperature drop.

I. Introduction

With the rapid advancement of Internet of Things (IoT) technologies, smart systems have proliferated across diverse environments. A smart home serves as a representative example of such sophisticated systems, operating through centralized platforms that integrate various IoT devices, including environmental sensors, actuators, and AI assistants. These systems are designed to enhance user convenience via automation mechanisms, such as the scheduled deployment of robotic vacuums for household maintenance or the implementation of automated shutdown protocols for air conditioners to optimize energy efficiency. To extend these capabilities, leading smart home platforms now host dedicated automation marketplaces [1], [2], [3], [4] featuring curated official applications.

Nonetheless, despite the operational conveniences of smart home configurations, emerging research has uncovered critical vulnerabilities in device-automation interdependencies [5], [6], [7], [8], [9]. As shown in Fig. 1, a heating unit scheduled to activate at 16:00 may elevate ambient temperatures above 27°C, which could inadvertently trigger secondary trigger-action programs (TAPs, or IFTTT-style **apps**) that initiate window ventilation protocols. Alternatively, smoke evacuation applications may conflict with thermal comfort maintenance systems due to overlapping window operations. Such unintended interaction chains can ultimately drive smart environments into states that contradict users' original automation intentions.

In response to these identified challenges, existing studies [6], [10], [11], [5], [7], [12], [13], [14] have primarily focused on mitigating these vulnerabilities through centralized analysis. Nevertheless, current research paradigms overlook users' proclivity to create custom automation apps [15], which exhibit broader functional diversity and heterogeneous design rationales. This oversight fundamentally limits the efficacy of existing systems [6], [10], [11], [5] in three critical dimensions:

First, contemporary conflict detection systems exhibit limited efficacy when processing apps with divergent design objectives. Current methodologies [7], [6], [12], [13], [14] employ rule-based detection heuristics predicated on predefined risk indicators. However, the determination of undesirable system states constitutes a subjective judgment that exhibits significant contextual variance across users and environmental

Manuscript received April 19, 2021; revised August 16, 2021.

Ronghua Li, Zi Liang, Yaxin Xiao, Qingqing Ye are with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong;(e-mail: cory-ronghua.li@connect.polyu.hk; zi1415926.liang@connect.polyu.hk; 20034165r@connect.polyu.hk; qqing.ye@polyu.edu.hk)

Haibo Hu is with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong and PolyU Research Centre for Privacy and Security Technologies in Future Smart Systems. (e-mail: haibo.hu@polyu.edu.hk;)

conditions. For instance, window-opening automation may represent either a security vulnerability or a benign operation, depending on whether occupants are present.

Second, most existing systems focus on conflicts between two apps, ignoring complex device-app or human-app interactions. For example, an app that opens windows for ventilation may conflict with a manual operation that closes windows for security reasons. Current detection frameworks [8], [6], [10], [5] do not account for such user-initiated actions, leading to incomplete conflict identification.

Third, predominant approaches lack comprehensive static conflict resolution (SCR) frameworks that balance adaptability with feasibility. Effective SCR requires providing prescriptive guidance for automation modification. However, current implementations demonstrate fundamental limitations: some merely flag conflicts without actionable modification strategies [16], [17], while others propose overly simplistic modification results [7], [6], [18], [12] that neglect user preference heterogeneity. For example, existing systems may recommend complete app removal [11] without considering functionality preservation requirements.

To address these limitations, we present PaCR (Preference-aligned Conflict Resolution), an LLM-agentic framework that iteratively refines automation apps while resolving conflicts through preference-aware optimization. Specifically, this work proposes the first **Agentic Conflict Handling framework**. While LLMs demonstrate strong capabilities, their direct application to conflict resolution tasks remains challenging [19], [20]. To address this, we implement a stepwise interaction process for agents: (i) It first analyzes existing smart home configurations and queries a simulated "user" agent to learn their design rationales and smart home preferences; (ii) The system then applies a specialized agentic workflow to leverage these insights for conflict detection and resolution.

Our solution presents two fundamental innovations: (1) We propose a structured preference context that captures user expectations through preference articulation, enabling personalized conflict mitigation with minimal degradation in utility; (2) We integrate conflict-resolving agents into pipelines, harnessing their semantic reasoning capabilities for device-interaction analysis and context-aware solution generation. This significantly reduces the cognitive burden on users when acknowledging conflicts and implementing resolutions.

The rest of the paper is organized as follows. In Section II, we introduce the background, large language models, and definitions of conflicts. In Section III, we analyze the advantages and disadvantages of related work. Section IV presents our rationale for resolving conflicts in a personalized manner and details the design of PaCR. We evaluate PaCR's system performance in Section V and conduct relevant discussions in Section VI.

II. Background

A. Smart Home Automation

Smart home technology has evolved rapidly over the past few decades, transforming modern living environments through centralized platforms [2], [4], [3]. These platforms interconnect IoT devices, enabling users to monitor device states and remotely control actuators. Users can configure trigger-action programs (TAPs/IFTTT rules) to automate device operations based on predefined conditions (e.g., activating air conditioning when the temperature exceeds specified thresholds). Each automation app consists of three components: (1) a trigger event, (2) conditional checks, and (3) executable actions. When the triggered conditions are met, the app executes the designated operations. Throughout this work, we use the term "**app**" to refer to these automation rules consistently.

B. Large Language Model

The advent of ChatGPT [21] has driven the widespread adoption of large language models (LLMs). Trained on extensive datasets, these models exhibit strong capabilities in contextual understanding and response generation. To utilize LLMs effectively, structured prompts must be designed using techniques such as contextual framing, few-shot examples, and chain-of-thought reasoning [22], [23], [24]. Our framework leverages these prompt engineering methods to harness LLMs' semantic understanding for analyzing app-device interactions and identifying potential conflicts.

C. Conflict Definitions

We define conflicts as undesirable behaviors of smart devices caused by interactions between automation logic and devices. These include: **I**. Unwanted competing control over the same devices (**Device Control Conflict, DCC**); **II**. Counteracting or mitigating influences on physical environments (**Property Influence Conflict, PIC**); and **III**. Inappropriate interactions among automation rules that contradict users' original expectations (**Interaction Source Conflict, ISC**). These three high-level conflict types encompass all previously identified conflicts [10], [6], [8] and additionally include a new category of user-app conflicts. We elaborate on the detailed definition of each conflict type in Section IV-E. Furthermore, since intra-app logical errors have been extensively studied [25], [26], [6], this work focuses on the detection and reasoning of inter-app conflicts, thereby complementing existing intra-app conflict resolution approaches.

III. Related Work

Automation conflicts have emerged from research on smart home security. Studies by [6], [5] first identified a new type of threat caused by conflicts or interactions between automation rules. This threat occurs when two apps compete for control over the same device, or when the functionality of one app unexpectedly interacts with the triggers, conditions, or actions of another app. Following this research trajectory, [10], [14], [13], [11], [27], [9], [8] have proposed both static and dynamic approaches to detect and resolve such conflicts. While this work shares similar conflict detection and resolution objectives, we aim to address several persisting limitations, as outlined below.

A. Conflict Detection

Previous conflict detection systems rely on either offline static analysis or online dynamic analysis. Static conflict detection (SCD) systems generally use natural language processing (e.g., embedding similarity) and other software algorithms to analyze relationships between devices and apps [7], [16], [6], [5], [12], but they suffer from high false-positive rates due to inaccurate semantic analysis when using naive models such as Word2Vec [28]. For example, a plug indicator light might be incorrectly associated with illuminance or brightness. To address this issue, [10], [13] proposed dynamic conflict detection (DCD) to validate static analysis results in real-world environments. However, despite its accuracy, this approach has fundamental limitations. First, it is time-consuming (e.g., [13] reports an average detection time of two hours) and inevitably disrupts users' routines by requiring repeated device cycling. Second, dynamic analysis provides only one-time validation, while environmental conditions like temperature and humidity evolve continuously. This temporal variability can alter detection accuracy over time. Some works, such as those by Xing et al. [29], [30], employ large language models (LLMs) to label relationships and construct graph neural networks (GNNs) that capture conflicting relationships between components. A key limitation of these works, however, is that conflicts labeled by LLMs often deviate from human perception—raising questions about their real-world utility and effectiveness. Therefore, training or fine-tuning a model to address this discrepancy can easily reduce its generality and adaptability to most users.

To achieve reliable conflict detection while maintaining efficiency, this work adopts static analysis enhanced with LLMs to reduce false positives. As demonstrated by [31], LLM-based prompt engineering with contextual embeddings effectively resolves semantic inaccuracies, significantly improving the performance of static analysis methods [5], [11], [12]. Additionally, we incorporate user preferences to ensure the authenticity of detected conflicts.

B. Conflict Resolution

Conflict resolution methods are equally important as detection and are also categorized into dynamic and static approaches. Dynamic methods typically use third-party hardware hubs or cloud servers to mediate communication between devices and platforms during runtime. However, most vendors implement closed-source systems, which limits cross-platform adaptability [13], [14]. IoTMediator [10] connects devices to a centralized hub that simulates virtual devices to enable platform integration. Nevertheless, its hardware requirement creates deployment barriers for average users. Static approaches offer broader adaptability by providing automation modification suggestions (e.g., adding or removing conditions) to preemptively mitigate risks. This enables conflict management even in closed-source platforms. Despite their cross-platform advantages, existing static solutions [5], [11], [6] generate generic modifications that disregard user personalization, or delete risky actions entirely [12]—reducing system utility [11]. Furthermore, involving users in the resolution step introduces additional uncontrollable variance, as users themselves are prone to making errors. To our knowledge, no current static method effectively resolves conflicts while preserving functionality. Moreover, requiring users to identify conflicting scenarios also imposes significant cognitive burden.

C. Language Models and Agents for Smart Homes

Smart home automation generation has emerged as a critical research area for enhancing system utility and user experience. Studies by [26], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41] have explored translating user descriptions or event traces into structured automation rules. However, training specialized models for this task requires substantial resources. Recent trends instead utilize pre-trained LLMs with prompt engineering [21], [42], [43]—an approach aligned with our use of LLMs' reasoning capabilities for conflict resolution.

In the context of smart home service provision, we also observe the rise of LLM-powered agents [44], [45], [46] that autonomously perform tasks through iterative reasoning and tool usage. Works such as [47], [48] apply LLM agents for natural language-based smart home control. However, these systems primarily focus on executing direct user commands rather than resolving conflicts. Our work pioneers the integration of LLM agents into a structured workflow specifically designed for personalized conflict detection and resolution in smart home environments.

D. Beyond Automation Conflicts

IoT systems also present numerous security and privacy challenges that require investigation. Works by [49], [50], [51] focus on detecting device anomalies and alerting users to unexpected scenarios. Other studies, such as [52], [53], [54], [55], [56], address personal data leakage and develop privacy-preserving solutions for modern IoT systems.

IV. System Designs

A. Design Rationale

We use a toy survey example to illustrate why user preference is important for personalized conflict detection and resolution. Suppose that APP1 "opens the window when smoke is detected" and APP2 "closes the window when the temperature drops below 26 degrees Celsius." When smoke is detected and the outdoor temperature drops, potential conflicts arise regarding control over the window. However, after surveying people offline with dozens of such seemingly conflicting pairs, we surprisingly noticed that some people consider the situations to be conflicting, while others do not care which event happens first, demonstrating significant variance in conflict interpretation and handling preferences. Due to such variance, we intend to incorporate users' own preferences into the system to ensure that the conflict handling results do not degrade system utility. Yet, there are several challenges to achieving such goals:

First, we find that users exhibit difficulties in comprehending potential conflicts, even if they are explained in natural language. Second, representing user preferences in a structured manner for conflict detection and resolution is non-trivial.

Users may struggle to articulate their expectations in a way that can be directly utilized by automated systems.

To this end, this work proposes an agentic refinement and conflict-resolving workflow to help users handle conflicts automatically. The core idea is to build an agent that can represent users to detect conflicts and address them in a nearly personalized manner. In the following sections, we first introduce the core steps of data processing, agent construction, and conflict handling. Additionally, since the agentic workflow demands a large amount of prompt engineering, we further introduce how we leverage meta-prompting techniques in Section IV-F to automatically refine all prompts for the agent and enhance the overall output quality.

B. System Overview

In this section, we propose PaCR, a **P**reference-**a**ligned **C**onflict **R**esolution framework (Fig. 2). It comprises three core steps: Knowledge Preparation (KP), Preference Enumeration (PE), and Conflict Handling (CH). In the first step (**KP**), PaCR analyzes the semantic contexts of smart home configurations (e.g., automation apps and device lists) (①) and portrays users via profiles, i.e., their living habits and characteristics (②). The main goal of KP is to build the knowledge base for the Conflict Resolving Agents (CRA). Since the automatically generated profiles may have only captured general characteristics, in the second step (**PE**), the CRA further enumerates potential user preferences for each of their automation apps to complement the comprehension of user preferences and rationales (③). Finally, in **CH**, the CRA leverages all app preference information to detect conflicts, address them, and apply fixing configurations to the smart home setting (④). In the following, we introduce the detailed designs of each step.

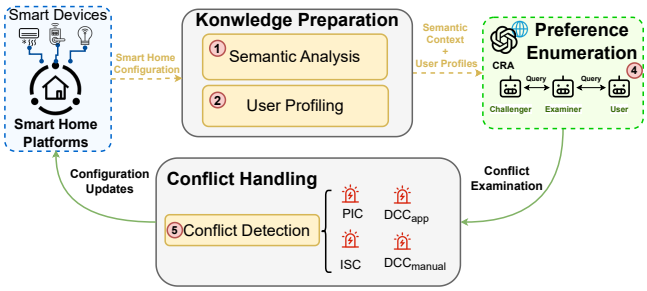

Fig. 2: Overview of PaCR framework.

C. Knowledge Preparation

KP serves two purposes: First, it collects basic information, such as the list of devices and existing automation apps, into the system and infers the functions of devices to assist subsequent inter-device/app interaction analysis. Second, it deduces the design rationales behind each app to profile users from the perspective of their smart home usage habits and preferences.

To begin with, PaCR analyzes the nature of devices, their capabilities, and their potential relationships with the environment or other devices, namely, Semantic Relationship Analysis (SRA). SRA aims to examine the relationships between these devices and 20 common physical properties[1], as shown in Fig. 3. However, as discussed in Section III, most existing SRA implementations rely on Word2Vec or n-gram models, which suffer from a high false positive rate. In addition, most LLM-assisted methods rely on comprehensive device descriptions from users, such as user manuals, which is very inconvenient for older devices. Given the recent proliferation of large language models (LLMs) and their enhanced reasoning capabilities, PaCR adopts LLMs for SRA, using purely device names and related automation apps. We first tested the baseline LLM-SRA, i.e., directly asking LLMs whether a device is related to another device over certain properties, and encountered a problem: the limited device configuration information (e.g., only device names/types) restricts the LLM's ability to fully comprehend device capabilities, potentially generating erroneous relationship analyses. To address this limitation, PaCR leverages a two step search: (i) it searches the device on Tasmota [57], an open-sourced IoT firmware platform that supports hundreds of IoT devices, for the sensing/actuation capabilities of the device. (ii) If exact match is not found, PaCR prompts LLMs to interpret function lists of similar devices, thereby enriching contextual information for subsequent analysis. In addition, all automation apps related to the device are also provided as context for the models to better understand their working scenarios and capabilities. Such enriched context improves the LLM's capacity to reason about device-property interactions. After employing Chain-of-Thought [23] and few-shot learning [22] techniques, hallucinations [58] in SRA barely existed during the experiments. To ensure the correctness of SRA results, PaCR further applies consistency check to ensure that all semantic relationships at least relate to one similar device in the database.

Next, as shown in Fig. 3, we formalize SRA results into intermediate data structures on a device-basis, similar to the "Scene Interaction" reference in [59]: each device will be linked to a set of properties that it interferes with or senses. During the subsequent conflict reasoning process, only entities with intersecting properties will be interpreted as "interactable and potentially conflicting" and processed to reduce computational overhead and improve accuracy.

After obtaining contextual information via SRA, PaCR uses LLMs to compile a comprehensive user profile for each app set, aiming to depict users from the perspective of design rationales, personal characteristics, and their potential interaction scenarios with the smart home environment. For instance, if a user has multiple apps related to temperature control, we may infer that the user is sensitive to temperature changes and prefers a comfortable indoor climate. This also applies to other aspects, such as security consciousness (e.g., multiple security-related apps), energy-saving preferences (e.g., apps related to power consumption), and convenience-oriented behaviors (e.g., apps automating routine tasks). By integrating these insights, PaCR constructs a user profile that encapsulates

[1]Properties are summarized from previous works, including *sound, temperature, gas, smoke, humidity, illuminance, air quality, carbon dioxide, infrared, ultraviolet, dust, power, energy, water, human voice, weather, acceleration, contact, presence, motion.*

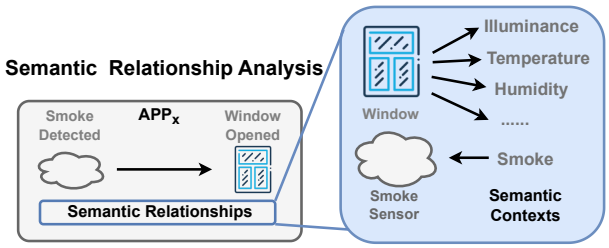

Fig. 3: SRA infers apps' and devices' semantic contexts with environmental properties.

user preferences and characteristics, serving as the knowledge foundation for the preference-aligned conflict resolution agent (CRA). While this cannot ensure a complete representation of user preferences, KP provides options for real users to review and adjust the generated profiles, thereby refining the system's understanding based on actual user feedback.

In summary, KP builds the knowledge foundation for the CRA by collecting smart home configurations, analyzing device-app-environment relationships, and profiling users based on their app design rationales and preferences. Such information is essential for the CRA to understand user preferences and effectively detect and resolve conflicts in subsequent steps.

D. Preference Enumeration

In the preference enumeration step, PaCR further strengthens the knowledge foundation for each automation app, i.e., *why each app is created*. More specifically, the strengthening process includes reasoning about why/when the app activates, how it influences the smart home, and when it should cease activation. Referencing these three aspects, PaCR implements more accurate conditions for activation, ensures the proper functionality of apps, and avoids unexpected condition enabling/disabling. However, since cultivating such information can be cognitively demanding for users, the CRA creates three sub-agents—the *Examiner*, the *User*, and the *Challenger*—to enumerate on these three perspectives. The idea is intuitive: the *User* refers to the user profile obtained from KP to fully represent the user; the *Examiner* asks the *User* questions to understand the design rationales behind each app; and the *Challenger* challenges the *User* with counterfactual scenarios to further explore potential missing conditions or unexpected interactions. The overall question framework is shown in Fig. 4, which includes three questions:

- **Direct Q:** How will the app be used? [*User* → *Examiner*]
- **Counterfactual Q1:** What if the app is triggered/conditioned by [*Challenger* examples]? [*Challenger* → *User* → *Examiner*]
- **Counterfactual Q2:** What if [*Challenger* examples] change the action states? [*Challenger* → *User* → *Examiner*]

Based on the dialogue contexts, the CRA summarizes the user's preference/intentions into a structured data piece as shown in Listing 1. Correspondingly, each part of the context is used to address one of the three types of conflict: the Activation context for ISC; the Functioning context for PIC; and the Deactivation context for DCC. In the following, we introduce the detailed construction process for the preference context.

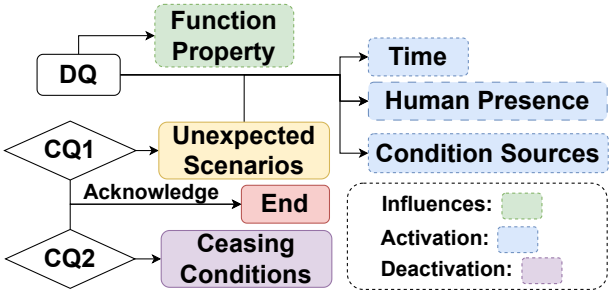

Fig. 4: Preference context construction workflow: *Examiner* analyzes *Users*' answers to infer the activation conditions, deactivation conditions, and influence of apps.

Listing 1: Preference Context Structure

```
Preference Context = {
  "Activation": {
    "Property Conditioning": [Src1,Src2,...],
    "Unwanted Conditioning": [Src1,Src2,...],
    "Temporal Conditioning":
        [Time1,Time2,...],
    "Presence Conditioning":
        [Cond1,Cond2,...],
  }
  ,
  "Functioning": {
    "Main":[Prop1,Prop2,...],
    "Side-Effects":[Prop1,Prop2,...]
  } ,
  "Deactivation": {
    "Sustaining": True/False,
    "Deactivation Conditioning":
        [Cond1,Cond2,...]
  }
}
```

1) DQ Reasoning: The CRA first directly analyzes the design rationales of each app to obtain a fundamental understanding of the Activation process and its conceptual/environmental influences. Specifically, the CRA uses two agents, an *Examiner* and a *User*, for the enumeration. The *Examiner* queries the *User* to describe the scenarios to which the app could be applied. By comparing the description with the original app configurations, the *Examiner* identifies the gap between expectations and actual designs, pinpointing potentially missing conditions—such as temporal conditions or human presence conditions—that can be added without degrading system utility. Additionally, the *Examiner* summarizes the conceptual/environmental influences of the app. Nonetheless, considering that the *User* may fail to disclose real users' subconscious intentions, the CRA further employs counterfactual reasoning to complement existing conditions and the logic of app reversion behaviors.

2) Counterfactual Reasoning on Unintended Interactions: The fundamental reasons for conflicts are the lack of proper constraints on app executions; i.e., apps can be triggered and can quit execution in more situations than expected. Therefore, the best way to disclose such problems is to directly exhibit counterfactual scenarios. To this end, the CRA uses the *Challenger* to generate counterfactual scenarios of the app to challenge the *User*. The generation process mainly follows three steps: (i) Imagine a scenario where all triggers/conditions could be satisfied; (ii) Substitute the triggering/conditioning sources or add new triggering/conditioning sources so that unsafe/energy-inefficient/insecure/meaningless events happen. Based on the responses from the *User*, the *Examiner* confirms missing constraints that could be included in the app to avoid unintended interactions.

3) Counterfactual Reasoning on Deactivation: Current smart home app configurations only support setting a device to a state but do not concern its subsequent behaviors. Yet, previous studies demonstrate that users have expectations regarding rules to "automatically change their action state when appropriate" [26]. As a result, it is important to reason about users' expectations regarding when to deactivate their apps. Accordingly, the *Challenger* generates several scenarios where the state of devices might be changed (manually or automatically) and queries the *User* about its considerations. The *Examiner* then extracts the *User's* opinions on the deactivation examples and converts them into a series of requirements (conditions). Depending on the type of requirement, apps are separated into "persistent" apps (action states are expected to be kept for a while) and "instant" apps (action states can be changed instantly).

After the preference enumeration step, the CRA obtains a comprehensive understanding of users' design rationales and preferences for each app, i.e., the preference contexts. Similarly, PaCR allows users to review and adjust the generated preference contexts for each app to reflect their intentions and preferences.

E. Conflict Handling

In the final CH step, the CRA identifies potential conflicts in the current smart home configurations and proposes resolutions accordingly, as shown in Fig. 5. As defined earlier, the CRA focuses on three types of conflict: PIC, ISC, and DCC, where DCC can be further classified into app-app conflicts (DCC_{app}) and app-user conflicts (DCC_{manual}).

1) Conflict Handling Principles: The CRA follows a safety/security-first principle when handling conflicts. Specifically, when conflicts involve safety/security concerns, the CRA prioritizes resolving these issues first to ensure the well-being of users and the security of their smart home environments. Additionally, the CRA will not add new constraints to such apps to maintain the generality of safety/security protocols. In contrast, for conflicts that do not involve safety/security concerns, the CRA aims to minimize utility degradation while addressing the issues. This means that the CRA will seek solutions that preserve as much of the original functionality and user preferences as possible (using as few additional conditions as possible), ensuring that the smart home system remains efficient and user-friendly even after conflict resolution. Since most conflicts are addressed via condition appending or action creation, the CRA needs to ensure that all newly generated entities follow the same syntactic format as existing apps. Fortunately, with the development of LLM backbones, models have demonstrated promising performance in following structural output formats. Therefore, for conflict resolution, the CRA specifies the output structure of trigger-action-programs via few-shot JSON examples.

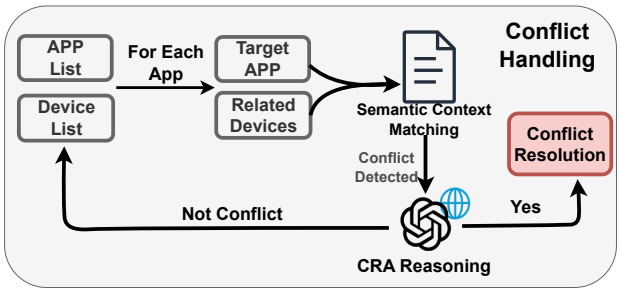

Fig. 5: Details of Conflict Handler. Semantic correlations are systematically examined to discover potential conflicts. CRA further reasons for such conflicts and applies resolution app modifications.

2) Detect and Handle PIC: PIC occurs when app actions or devices generate mutual interference, which may further lead to energy waste and unsatisfying living conditions. PaCR leverages **functioning properties** ($PROP_{function}$) of preference contexts to analyze whether an app causes countering effects on the functioning properties of other devices/apps. PaCR obtains the list of devices in the same scene as the app and applies semantic relationship verification to their correlations over the app's major environmental properties. Upon detection of any potential interferences, PaCR feeds the device and the app into the CRA for conflict reasoning and instructs the CRA to directly modify the app to avoid such conflicts.

Listing 2: PIC Resolution Prompt

```
You are a smart home expert who identifies
    unwanted interferences between devices
    and triggers action programs. You'll be
    given a tap and a list of devices that
    may have countering effects on the
    current app. Your job is to analyze
    whether such unexpected interference
    exists. If so, return true and add new
    conditions in the ''triggers'' to
    constrain the execution. Your return must
    stick to the original trigger action
    program JSON format like the following:

'''json
{
    "result": true/false,
    "app": {
        // Updated tap if interference exists
            (true); Otherwise leave this
            empty.
```